PROJECT BLUEPRINT · V1 · NOVEMBER 2026

AgentOS

An AI workforce for small service businesses. It starts with one job, done really well.

> **The one feature we build first:** when a lead comes in, AgentOS reads it, scores it and drafts a reply. A person approves it with one click, then the email goes out. Every step is logged. That's the whole MVP. Everything else in this document comes later.

What's in this document

#	Section	What it answers
1	Overview	What we're building now and what waits until later
2	Before you start	Accounts, tools and decisions you need on day one
3	Tech stack	What we use now, what we add later, and why
4	Architecture	Three simple diagrams: the workflow, the system and the lead states
5	Data model	The six tables the MVP needs
6	Feature roadmap	Every feature, in the order we add it
7	Build prompts	Ten copy-paste prompts for building the MVP with Claude Code
8	Definition of done	The checklist that says the MVP is finished

Who it's for

First customer: small digital marketing agencies (5–30 people) that lose leads because nobody replies fast enough or follows up. **Later:** consultants, education institutes and other service businesses.

Scope for v1

In v1 (build now)	Not in v1 (later phases)
Lead intake from a web form or API	WhatsApp, Instagram, calendar
AI extracts details, scores fit, drafts a reply	Quotations, automatic follow-ups
Approve / edit / reject screen	Multi-agent orchestration (LangGraph)
Email is sent only after approval	Long-term memory, RAG over documents
Simple lead board (status per lead)	External CRM sync (HubSpot, Zoho)
Audit log of every change	Billing, plans, analytics dashboards
One agency, a few users (built multi-tenant from the start)	Roles beyond owner / member

2 Before you start

Have these ready before writing any code. It takes about an hour.

Accounts

Account	Why	Cost to start
GitHub	Code, pull requests, CI	Free
Anthropic Console	Claude API key for extraction and drafting	Pay per use (a few $ during development)
Resend	Sends the email after approval	Free tier
Clerk	Login; each organisation becomes a tenant	Free tier
Railway or Render	Hosts the backend, Postgres and Redis	About $5–10 / month
Vercel	Hosts the Next.js frontend	Free tier
Sentry	Error tracking from day one	Free tier

Tools on your Mac

Tool	Version	Note
Node.js + pnpm	Node 22 LTS or newer	Your Node 26 works
Python via uv	3.12	Your system Python is 3.9, so let uv manage 3.12 for the backend
Docker Desktop	latest	Runs Postgres and Redis locally with one command
Git	2.39+	Already installed
Claude Code	latest	Runs the prompts in section 7

Three things to decide once

- **A first test agency** and 5–10 realistic sample leads. Fake but believable data makes the AI's work easy to judge.
- **The agency's facts sheet:** services, price ranges and turnaround times. In v1 this is a plain settings field, not RAG.
- **Reply tone:** friendly or formal, short or detailed. It goes into the prompt as a setting.

> **Rule for the whole project:** build the smallest thing that is correct, then add to it. If the current phase doesn't need a tool, don't install it.

3 Tech stack

This is based on your reference sheet, with fewer moving parts at the start. The "Add later" column says what gets added and when it's actually needed.

Area	Use now (MVP)	Add later	Why
Frontend	Next.js (App Router) + TypeScript + Tailwind + shadcn/ui	—	Fast way to build clean dashboards and forms
Backend	FastAPI + Pydantic v2	—	Typed APIs with built-in validation; Python suits the AI code
Database	PostgreSQL 16 + SQLAlchemy 2 + Alembic	pgvector (Phase 2)	One database for everything; vectors live in it too
AI	Claude API with structured outputs: Haiku 5.5 extracts, Sonnet 5.5 drafts	Prompt caching, evals	Output is checked against a schema before anyone sees it
Workflow	Explicit state machine in code	LangGraph (Phase 5)	Predictable, testable and resumable. LangGraph only when there are several agents
Jobs	Redis + arq worker	—	Retries and timeouts; slow AI calls stay out of web requests
Auth	Clerk (organisations = tenants)	SSO for bigger clients	Don't build login yourself
Messaging	Email via Resend	WhatsApp Business API (Phase 4)	Email is low-risk. WhatsApp needs Meta approval and message templates
Monitoring	Sentry + structured logs	OpenTelemetry (Phase 5)	See errors on day one; add tracing once there are many services
Dev / deploy	Docker Compose, GitHub Actions, Vercel + Railway	—	The same setup locally and in production
Automation	—	n8n: probably never	The workflows are the product, so they stay in our own code

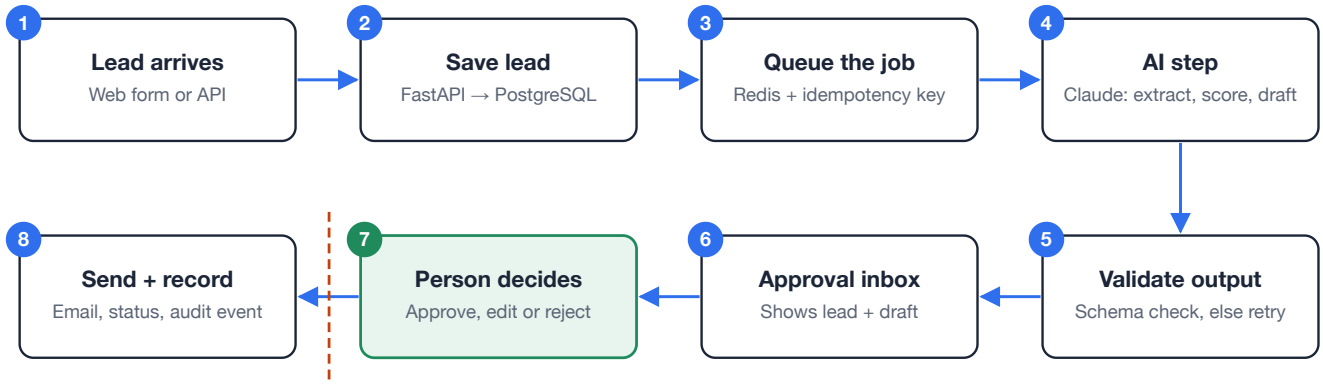
Repository layout

```
agentos/
  apps/web/            # Next.js frontend
  apps/api/            # FastAPI + worker (one codebase, two processes)
    app/routes/  app/models/  app/workflow/  app/ai/  app/jobs/
  docker-compose.yml   # Postgres + Redis
  CLAUDE.md            # rules for Claude Code
  docs/                # this blueprint + decision notes
```

4 Architecture

Three pictures explain the whole MVP. Read them top to bottom.

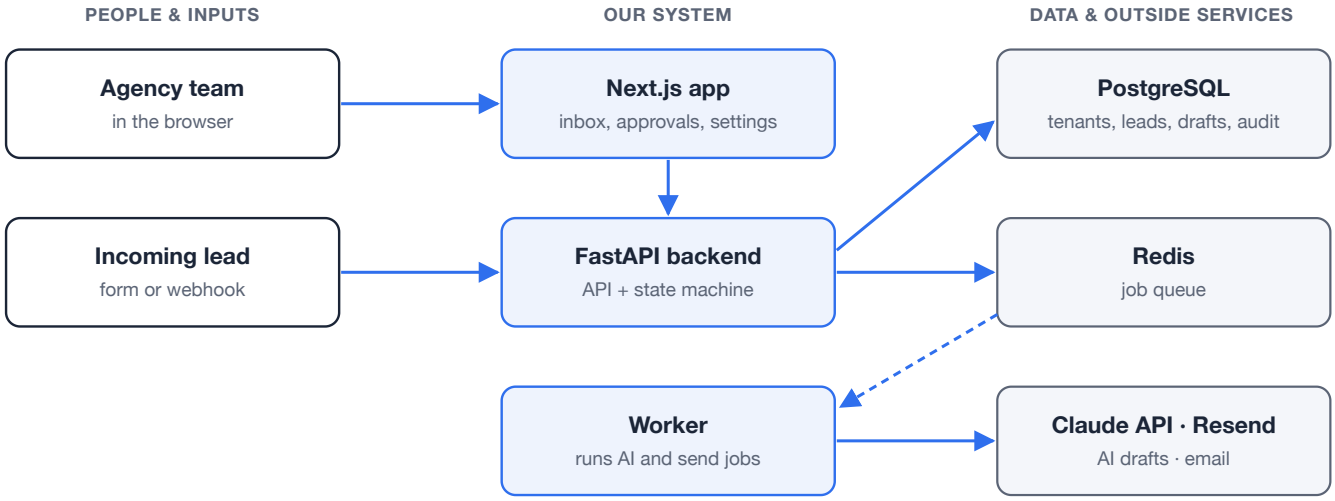
A. The workflow: what happens to one lead



SAFETY GATE: nothing leaves the system before step 7

Every arrow is a state change written to the audit log. Any failed step can be retried without duplicates.

B. The system: which part talks to which



The web app never calls Claude or sends email itself. It only changes a lead's state; the worker does the slow and risky work.

C. Lead states: the state machine



Transition	Who triggers it	What gets written
NEW → PROCESSING	The worker picks up the job	Audit event, attempt count
PROCESSING → DRAFT_READY	AI output passed validation	Extracted details, score, draft
PROCESSING → FAILED	Timeout, API error or invalid output, after 3 tries	Error reason; a person can retry it
DRAFT_READY → APPROVED / REJECTED	A signed-in user of the same agency	Who, when, and the final text if edited
APPROVED → SENT	The worker sends exactly once (idempotency key)	The provider's message ID

Five rules that keep it reliable

- **Tenant boundary:** every table has a `tenant_id` and every query filters by it. This is covered by tests.
- **Idempotency:** intake and sending both carry a unique key, so a double-submit or a retry never sends twice.
- **Allowed transitions only:** one function owns every state change. Anything outside the allowed list is rejected.
- **Validate AI output:** a Pydantic schema check runs first. Bad output is retried, then marked FAILED, and never reaches the user unchecked.
- **Append-only audit:** audit events are inserted and never updated or deleted.

5 Data model (MVP)

Table	Key columns	Purpose
tenants	`id, name, settings (facts sheet, tone)`	One row per agency
users	`id, tenant_id, clerk_user_id, role`	Who can approve (owner / member)
leads	`id, tenant_id, source, name, email, message, status, idempotency_key, created_at`	The enquiry and its current state
lead_analysis	`lead_id, intent, service, budget, urgency, fit_score, model, raw_json`	Validated AI extraction
drafts	`id, lead_id, body, edited_body, decision, decided_by, decided_at`	Suggested reply + the person's decision
audit_events	`id, tenant_id, lead_id, actor, action, from_state, to_state, data, at`	Permanent history of everything

Indexes: `(tenant_id, status, created_at)` on leads, and a unique `(tenant_id, idempotency_key)`. Each outgoing send gets its own unique key per draft.

6 Feature roadmap

Work one phase at a time. Don't start the next phase until at least one real agency is using the current one.

Phase	Features	New tech	Exit test
0 · Foundation week 1	Repo, Docker, database, auth, CI, a deployed "hello" page	—	You can sign in and see an empty inbox in production
1 · Lead reply MVP weeks 2–5	Lead intake · AI extract, score and draft · approval inbox · email after approval · audit log · retries	Claude API, Redis, Resend	All six acceptance criteria pass (section 8)
2 · Knowledge	Upload services, pricing, FAQs and case studies · drafts grounded with RAG, with sources shown	pgvector	Drafts quote correct prices from the agency's own documents
3 · Follow-ups & quotes	Scheduled follow-ups (still need approval) · quotation draft as PDF · reminders to the owner	Scheduled jobs	No lead goes 48 hours without a reply or follow-up
4 · Channels & CRM	WhatsApp Business · Gmail / Outlook sync (OAuth) · HubSpot / Zoho sync · webhook signature checks	Meta API, OAuth	A WhatsApp lead is handled end to end
5 · Platform	Roles (owner / manager / agent) · analytics: response time, conversion, time saved · billing · multi-agent flows	LangGraph, OpenTelemetry, Razorpay	Paying customers sign up and pay without your help
6 · Future	Calendar booking · document workspace · long-term customer memory · new industries (consulting, education)	As needed	Chosen by customer demand

> **Prices are guesses for now.** The tiers to test are ₹999, ₹2,999 and ₹7,999 a month. Ask 10 agencies what they'd pay during Phase 1, before building billing.

Skills you'll learn along the way

Phase	Skill
0–1	Schema design, transactions, queues, retries, idempotency, structured LLM output, multi-tenant boundaries, human-in-the-loop approvals
2	RAG: chunking, embeddings, retrieval quality, grounding
3–4	Scheduling, OAuth scopes, token storage, webhook verification, secrets management
5	Role-based access, product analytics, billing, tracing, agent orchestration

7 Step-by-step build prompts

Run these in Claude Code in order, one per branch. Don't move on until the **Done when** line is true, and commit after each prompt. Save this file as `docs/blueprint` in the repo so the prompts can point to it.

PROMPT 0 Project rules

```
Create CLAUDE.md for AgentOS, a multi-tenant SaaS that helps small marketing agencies reply to leads. MVP flow: lead intake -> AI extract/score/draft -> human approval -> email sent -> audit event. Stack: Next.js + TypeScript + Tailwind + shadcn/ui (apps/web); FastAPI + Pydantic v2 + SQLAlchemy 2 + Alembic (apps/api); PostgreSQL; Redis + arq; Claude API; Resend; Clerk. Rules: every table has tenant_id and every query filters by it; no external message is sent before approval; state changes only go through one transition function; audit_events are append-only; keep code minimal.
```

Done when: CLAUDE.md states the stack, the flow and the rules.

PROMPT 1 Scaffold

```
Scaffold the monorepo described in CLAUDE.md: apps/web (Next.js App Router, TS, Tailwind, shadcn/ui), apps/api (FastAPI managed by uv, Python 3.12), docker-compose.yml with Postgres 16 and Redis 7, .env.example, and a Makefile with dev, test and migrate targets. Add GET /health to the API and a home page that calls it. Add a GitHub Actions workflow that lints and tests both apps.
```

Done when: `make dev` starts everything and the home page shows "API: ok".

PROMPT 2 Database + state machine

```
Add SQLAlchemy models and the first Alembic migration for tenants, users, leads, lead_analysis, drafts and audit_events (columns as in docs/blueprint section 5). Lead status enum: NEW, PROCESSING, DRAFT_READY, APPROVED, REJECTED, SENT, FAILED. Unique index on (tenant_id, idempotency_key); index on (tenant_id, status, created_at). Write app/workflow/transitions.py with an ALLOWED map and transition(lead, to_state, actor, data) that updates status and inserts an audit_event in the same transaction. Add pytest tests for allowed and rejected transitions.
```

Done when: migrations run and the transition tests pass.

PROMPT 3 Auth + tenants

```
Integrate Clerk in apps/web with organisations. In apps/api, verify the Clerk JWT, map the org id to a tenant and user, and expose a get_current_tenant dependency used by every route. Add a test proving a user from tenant A gets 404 for tenant B's lead.
```

Done when: the cross-tenant test passes and signed-out users are sent to login.

PROMPT 4 Lead intake

```
Add POST /v1/leads, authenticated by a per-tenant intake API key, accepting name, email, company, message, source and an Idempotency-Key header. A repeated key returns the existing lead instead of creating a new one. Save as NEW, write an audit event and enqueue the arq job "process_lead". Add a simple test form page in apps/web for submitting sample leads.
```

Done when: submitting the same lead twice creates one row and one job.

PROMPT 5 AI step

```
Create app/ai/ with two functions using the Anthropic Python SDK and structured outputs: extract_lead(lead) -> LeadAnalysis (intent, service, budget, urgency, fit_score 0-100, reasoning) and draft_reply(lead, analysis, tenant_settings) -> Draft. Validate both with Pydantic. Keep prompts in separate files. Use claude-haiku-5-5 for extraction and claude-sonnet-5-5 for drafting. Test with recorded fake responses (no live API calls in CI).
```

Done when: the tests pass and a manual run on 5 sample leads gives sensible output.

PROMPT 6 Worker + retries

```
Implement the arq job process_lead: NEW -> PROCESSING, call extract_lead and draft_reply, save the results, then -> DRAFT_READY. Timeout 60s, max 3 attempts with backoff; after the last failure -> FAILED with the error stored. The job must be safe to run twice (skip if already past PROCESSING). Add POST /v1/leads/{id}/retry for FAILED leads.
```

Done when: killing the worker mid-job and restarting it finishes the lead with no duplicates.

PROMPT 7 Approval inbox

```
Build apps/web pages: /inbox (lead list with status badge, fit score and time since arrival; filter by status) and /leads/[id] (lead message, extracted details, editable draft, Approve and Reject buttons, audit timeline). Add the API routes they need: GET /v1/leads, GET /v1/leads/{id}, POST /v1/drafts/{id}/approve (optional edited body), POST /v1/drafts/{id}/reject (reason). Keep the layout clean and simple with shadcn/ui.
```

Done when: a user can approve or reject in the browser and the timeline updates.

PROMPT 8 Send only after approval

```
On approval, enqueue send_reply. The job sends through Resend with an idempotency key built from the draft id, stores the provider message id, and moves APPROVED -> SENT. It must refuse to send unless the lead is APPROVED. Add a test that tries to send a DRAFT_READY lead and asserts nothing is sent.
```

Done when: approved replies reach a test inbox and unapproved ones never do.

PROMPT 9 Acceptance tests + deploy

```
Write an end-to-end pytest covering the six acceptance criteria in docs/blueprint section 8. Add Sentry to both apps. Deploy apps/web to Vercel and apps/api + worker + Postgres + Redis to Railway, running migrations on deploy. Write a short README covering setup, environment variables and how to run the tests.
```

Done when: every acceptance test passes in CI and the production URL works end to end.

8 Definition of done (MVP)

#	Acceptance criterion	Proven by
1	A test lead can be submitted and is stored exactly once	Prompt 4 idempotency test
2	The model returns validated structured details and a draft	Prompt 5 schema tests
3	Only an authorised user of that agency can approve or reject	Prompt 3 cross-tenant test
4	No external message is sent before approval	Prompt 8 send-guard test
5	Every state change appears in the audit log	Transition tests + timeline screen
6	Failures can be retried without duplicate actions	Prompt 6 restart test

After the MVP: the first 30 days

- Give it to 3 agencies for free and watch them use the inbox.
- Track three numbers: time from lead to reply, the share of drafts approved without edits, and leads handled per week.
- Choose Phase 2 or Phase 3 based on what they complain about most, not on what's most fun to build.

> **One sentence to remember:** people trust AgentOS because a human approves every message and every action is logged. Protect that, and the features can grow safely.